



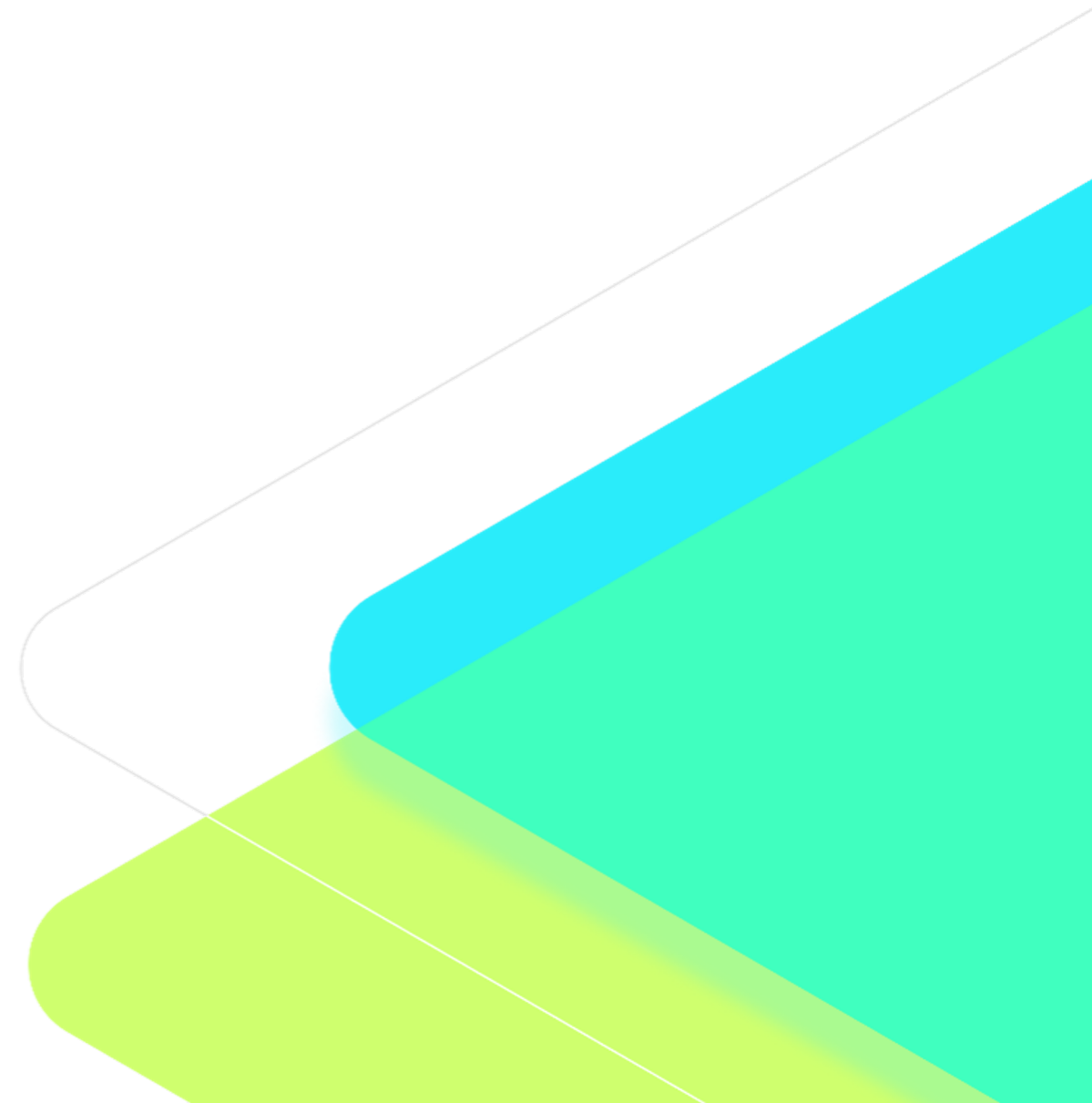
Vector stores and scalable similarity search



Deepali Srivastava

Author

[linkedin.com/in/deepali-srivastava/](https://www.linkedin.com/in/deepali-srivastava/)



What are vector stores and why do you need them?



Manual similarity search using embeddings

```
query_vector = embeddings.embed_query(query)
doc_vectors = embeddings.embed_documents(documents)

for doc, vec in zip(documents, doc_vectors):
    score = cosine_similarity(query_vector, vec)
    results.append((score, doc))

results.sort(key=lambda x: x[0], reverse=True)

for score, doc in results[:3]:
    print(f"{score:.3f} -> {doc}")
```

Compare the query with every document using cosine similarity

Sort results to retrieve the top-K most relevant documents



Scaling challenges in similarity search

Works for small data, but real systems have millions of documents

Comparing with every vector ($O(N)$) is very slow

Example: 1M documents $\times$ 768 dimensions $\approx$ 768M values per query

Large-scale data needs efficient storage and faster search systems



Why you need vector stores



Efficient storage

Need efficient storage for embeddings with metadata



Fast search

Need fast similarity search without checking every vector



What is a vector store?

Specialized system for storing embeddings and performing efficient similarity searches

- Stores vectors
- Builds indexes
- Performs similarity searches



Indexing for efficient search

Indexing narrows search to relevant regions instead of all vectors

Approximate nearest neighbor (ANN) searches a small subset of likely similar vectors

K-nearest neighbor (KNN) checks every vector using brute-force search



KNN vs. ANN in similarity search



KNN

Checks all vectors

Results are 100% accurate

Slow for large datasets

Scalability: poor



ANN

Checks only a subset

Slightly approximate (~95–99%)

Very fast for large datasets

Scalability: excellent

Tradeoff is acceptable



Types of ANN indexing approaches



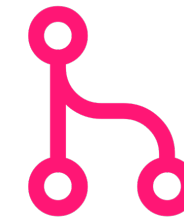
Clustering-based

Groups similar vectors,
e.g., inverted file index
(IVF)



Graph-based

Connects vectors as a
network, e.g., hierarchical
navigable small world
(HNSW)



Tree-based

Splits space into yes/no
branches, e.g., annoy



Working with vector stores using LangChain



From embeddings to vector stores in LangChain

No manual
embedding calls using
`embed_query()` or
`embed_documents()`

Embedding model
is passed to the
vector store

LangChain connects
model and
vector store





Vector stores

- Chroma
- FAISS
- Pinecone
- Weaviate

Pinecone and Weaviate offer cloud and API-based features

FAISS and Chroma are lightweight and used locally

Choose vector store based on use case

All vector stores store embeddings and enable similarity search



Unified interface with LangChain



Different vector stores have different APIs



All support common operations like add, search, delete



LangChain provides one unified interface



`add_documents()` : Add data to the store

`similarity_search()` : Find similar documents

`delete()` : Remove documents by ID



Working with Facebook AI Similarity Search (FAISS) for local vector search



Demo

Building a vector store with FAISS



Demo

Loading a vector store from disk



Working with Pinecone for scalable vector search



Demo

From files to vector store



Performing similarity search with different distance metrics



Demo

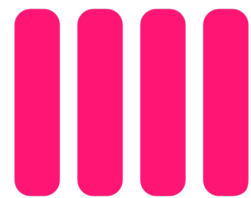
Interpreting scores in Pinecone



Configuring and choosing metrics



Pinecone → set metric using the metric parameter
(cosine, dot product, euclidean)



FAISS → metric chosen via index type
(IndexFlatL2 for distance, IndexFlatIP for inner product)
Normalized vectors make inner product similar to cosine similarity



Match metric with embedding model



Metadata filtering and hybrid search



Limitations of semantic search

Documents across HR,
finance, legal, and
technical domains

Query: “Latest leave
policy for employees in
India”

Semantic search
retrieves meaning but
may miss exact
requirements

Embeddings do not
capture structured
constraints

Similar results are
not always the
correct results



Improving retrieval with metadata filtering



Documents include metadata like country, department, and year



Filters are applied before similarity search



Narrows search to the most relevant subset



Improves accuracy, relevance, and efficiency



Demo

Metadata filtering for better search



Another limitation of semantic search

Semantic search may miss exact keywords, IDs, etc.

Query: “Fix error code
ERR_CONN_RESET_4521”

Returns general results instead of
exact matches

Exact tokens require keyword-
based search



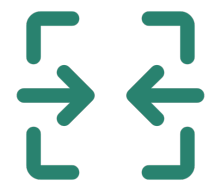
Hybrid search for meaning and precision



Combines semantic similarity and keyword matching



Ranks results using combined scores



Uses dense vectors and sparse representations like BM25



Useful for queries with exact terms and semantic intent



Combining metadata filtering and hybrid search



Metadata filtering

Enforces structured constraints and narrows the search space



Hybrid search

Ranks results using semantic meaning and exact keyword matching



Maximal marginal relevance (MMR) for diverse retrieval



Limitation of similarity search

Returns top K results based only on similarity

Highly similar documents lead to repetitive and redundant results



Redundant results in similarity search

"Vector databases are used in search engines"
"Vector databases improve semantic search"
"Vector databases power modern search systems"
"Vector databases are used in recommendation systems"
"Vector databases are applied in fraud detection"
"Vector databases detect anomalies in transactions"
"Fraud detection systems use vector databases"
"Vector databases enable similarity search for images"



"Applications of vector databases"

"Vector databases are used in search engines"
"Vector databases improve semantic search"
"Vector databases power modern search systems"



Redundant results

"Vector databases are used in search engines"
"Vector databases are used in recommendation systems"
"Vector databases are applied in fraud detection"



Diverse results



Redundancy reduces RAG performance



Token wastage

Redundant results send repetitive information to the LLM, wasting tokens



Less information coverage

Limited information coverage reduces answer completeness



Maximal marginal relevance for diverse retrieval

MMR is a re-ranking technique that balances relevance and diversity

Improves context quality for better LLM answers in RAG systems



How to Use MMR in LangChain

Use the method `max_marginal_relevance_search()`

`k`: Final number of results returned

`fetch_k`: Number of initial documents retrieved

```
results = vectorstore.max_marginal_relevance_search(  
    query,  
    k=3,  
    fetch_k=10  
)
```



Choosing fetch_k for MMR

fetch_k must be greater than
k to enable diversity

Larger candidate pool
allows better selection of
diverse results



Tuning MMR with `lambda_mult`

`lambda_mult = 1` → Prioritizes relevance

`lambda_mult = 0` → Prioritizes diversity

`lambda_mult = 0.5` → Balanced

```
results_mmr = vectorstore.max_marginal_relevance_search(  
    query,  
    k=3,  
    fetch_k=6,  
    lambda_mult=0.5  
)
```



Demo

MMR for diverse search results



From naive search to production retrieval

KNN: focuses on
relevance only

ANN: improves speed
and scalability

Metadata filtering:
narrows search to
the right subset

Hybrid search:
combines semantic
and keyword search

MMR: re-ranks results
for diversity

